SECTIONS

0. The short version
1. The entry in the chat header
2. Setup state: how the client knows AI is available
3. Summarise
4. Share in chat
5. Privacy and end-to-end encryption
6. Other AI actions inside the chat
7. Feed, notifications, unread, persistence
8. Auth and permission rules for mobile
9. Test data
10. Recommendations for mobile

connectum/docs/AI_CHAT_FOR_MOBILE.md

Copy markdown

AI inside a chat window: how it works today (for the mobile team)

Written from the code, nothing changed. Paths are relative to `connectum/` unless stated. Scope: only what lives **inside an open conversation**. The AI tab / My Workspace assistant (`AssistantView.jsx`, `ai.conv.*`) and agent management are out of scope except where the chat feature depends on them. Read as of 10 Nov 2026 on branch `feat/smartview-fix`; line numbers drift, search by the quoted text. Anything not proven by the code is marked **UNSURE**; anything absent is marked **not implemented**.

Wire format for every call below (same as `TASKS_AND_ACTIONS_FOR_MOBILE.md`): send `/sd <action> {json,"reqId":n}` on the chat socket; the reply is `{"type":"sd","action","reqId","ok":true,"data":{...}}` or `{"ok":false,"error":{"code","message","details"}}`.

---

0. The short version

- The web has **exactly one** AI feature inside a chat: **"Summarise this chat with AI"** (a button in the chat header that opens a window). Care threads (`room-care-*`) have a second, different summary that is **not** AI-written.
- There is **no server action that summarises**. The browser reads the chat history itself, builds the prompt, and sends it to the organisation's AI provider through a generic relay

(`assistant.relay`). The server only adds the API key and forwards one request. The answer is **one non-streamed response**.

- The result is shown **only on the screen of the person who asked**. It is **never posted to the chat**, never stored, has no id, and cannot be fetched again. A **Copy** button is the only way out.
- There is **no consent text and no block in end-to-end encrypted chats**: the app decrypts on the device and sends the plain transcript to the provider through the server (section 5). This is the most important thing for mobile to decide on.

---

1. The entry in the chat header

Source: `chat-app/features/connectumChat/components/ConversationHeader.jsx:46-70` , wired in `components/ChatScreen.jsx:281-291` and `ConnectumChatScreen.jsx:1570-1571` .

1.1 What is shown

CHAT	HEADER BUTTON	ICON / TOOLTIP	OPENS
Direct chat (`user-<username>`)	yes	`summarize` / "Summarise this chat with AI"	the **Summarise this chat** window (section 3)
Group (`room-<name>`)	yes	same	same
Host chat (`host-<name>`)	yes	same	same (reads with scope `host`)
General broadcast (`room-general`)	yes	same	same (reads with scope `global`)
Care thread / patient thread (`room-care-<person>`)	**different button**	`auto_awesome` / "Summary of this thread"	the **thread summary card** (section 6.1), not the AI window
My Workspace (`workspace`)	no header at all (`inConversation` is false, `ChatScreen.jsx:219`)		
Ticket / record conversation	**UNSURE** : it is shown as a normal chat; the code has no special case, so the same button appears unless its id starts with `room-care-` .		
Reply thread under	**not implemented** (no AI on		

one message	a message or sub-thread)
End-to-end encrypted chat (direct or group)	yes, **no difference**. The lock badge (`E2EBadge`) sits next to it but does not hide or change the AI button

1.2 Who can see it

- **Everybody who has the conversation open.** There is no role, permission or feature-flag check on the button. `ConnectumChatScreen.jsx:1571` always passes the handler, and `ConversationHeader` only tests "not a care thread".
- The button is shown **even when no AI is set up**. The window then tells the person (section 2). A mobile client may choose to hide it (section 10).

1.3 What it opens (menu)

There is **no menu**. One tap opens the window directly: `setModalParam({ chatId }); setModal("chat_summary")` . Nothing to list in order.

Other ways the web opens the same window:

TRIGGER	RESULT
Typing `#<agent command> chat summary` in the composer (for example `#claude chat summary`)	window opens for the open chat (`routeHashCommand.js:249-255`)
`#<agent command> chat summarise since 2026-10-01` (also `summarize` , `from`)	same, with the start date preset. Regex: `^chat\s+summar(?:y

`<agent command>` is the agent's `alias` (section 2). Any other text after `#<alias>` opens the AI tab and asks the question there (out of scope).

---

2. Setup state: how the client knows AI is available

2.1 The call

```
/sd ai.agents {"reqId":11}
```

Access level: `user` (any signed-in person) (`core/sd_cmds.erl:201` , `:603`). Answer (`integrations/sd_assistant.erl:59-65`):

```
{"type":"sd","action":"ai.agents","reqId":11,"ok":true,"data":{"agents":[
  {"id":"claude","name":"Claude","alias":"claude","provider":"anthropic","model":"claude-haiku
```

```
]}}
```

- Only agents that are **enabled** and that **this person is allowed to use** are returned. Never a key or an address.
- The AI agent plugin may add extra fields per agent (`sd_plugin_host:agent_view` , for example roles). **UNSURE** of the exact names; ignore unknown fields.
- The web caches the list (`workspaceAgent.js:49-57`) and refreshes it when the server pushes `ai_agents_changed` :

```
{"type":"sd_event","event":"ai_agents_changed","data":{"action":"saved","by":"admin"}}
```

`action` is `saved` or `deleted` ; it goes to everyone online. Re-call `ai.agents` on it and after every reconnect (events sent while offline are lost).

2.2 The states, and what the client can tell apart

There is **no single "AI status" flag**. The state is derived:

STATE	HOW IT SHOWS	WEB TEXT
Set up and allowed	`agents.length >= 1`	window works. With 2+ agents an "AI" chip row appears to choose one; the first is selected by default
Not set up / disabled / no permission	`agents` is `[]` (the server does **not** say which of the three)	`No AI is set up for you yet. Ask your administrator.` (a note in the window, `ChatSummaryModal.jsx:68`); the Summarise button is disabled
Agent removed or revoked between list and call	`assistant.relay` fails with `not_found` or `no_access`	`That AI agent no longer exists.` / `You do not have access to this AI agent. Ask your administrator.` The web also drops its cached list
Key invalid	provider answers 401/403, or relay `key_unreadable`	`The AI provider no longer accepts this key (it may have been rotated, revoked or expired). Add the new key to continue.` / `The stored key could not be read. Add it again.` (`services/assistantErrors.js`)
No credit / quota exceeded at the provider	provider answers 402, or 4xx with "insufficient quota / billing / credit" in the body	`Your AI provider account has no credit left. Add credit on the provider's billing page, then ask again. Your key is fine.`

Rate limited	server `rate_limited` (30/min per person, 600/min per agent) or provider 429	`Too many questions in a minute. Wait a little and try again.` / `The provider is limiting how fast you can ask. Wait a minute and try again.`
Connectum has a per-user or per-org AI quota	**not implemented** (only the two rate limits above; no token or credit meter)	
An org-wide on/off toggle for AI in chat	**not implemented** (the only switches are per agent: `enabled` and who may use it)	

Note for `pick_agent` (`sd_assistant.erl:254-268`): if `assistant.relay` is called **without** `agent` , it works only when the person may use exactly one agent; with none it answers `no_agent` , with several `invalid` "Choose which AI agent to ask."

2.3 Who can set it up, and where

- **Administrators only**: Admin console > **AI agents** (`AdminConsoleModal.jsx:86` , panel `modals/AdminAiPanel.jsx`). Actions `admin.ai.list` , `admin.ai.save` , `admin.ai.delete` , `admin.ai.test` are access level `admin` (`core/sd_cmds.erl:163`). Keys belong to the organisation; nobody gets a key back after saving it.
- Who may **use** an agent is a permission named `ai:<id>` (Admin console > Permissions; `sd_assistant:may_use`). Administrators always can.
- A **non-admin** sees only the note `No AI is set up for you yet. Ask your administrator.` The web has no "request AI" action. **not implemented** .

`admin.ai.save` arguments (`sd_assistant.erl:75`): `id?` , `name` (1-40), `alias` (command, 2-20 chars `^[a-z][a-z0-9_-]{1,19}$` , not a reserved word such as `ai` , `me` , `help` , `ask` ...), `provider` (`openai` , `anthropic` , `gemini` , `openrouter` , `grok` , `groq` , `deepseek` , `sambanova` , `custom`), `model?` , `baseUrl?` (required for `custom` , public http(s) only), `key` (8-400 chars, no spaces), `enabled?` , `modelChoice?` , `access?` `{all|users|departments|designations|categories}` . Maximum 12 agents. The id of a new agent equals its alias.

Error messages the admin sees on save include: `Give the agent a name (up to 40 characters).` , `That command is already used by something else. Choose another.` , `Another agent already uses that command.` , `The organisation has reached the number of agents it can have.` , `Paste the full API key (8 to 400 characters).`

---

3. Summarise

3.1 What the web does, step by step

Source: `modals/ChatSummaryModal.jsx` , `services/workspaceAgent.js` , `services/chatReader.js` .

1. Open the window; call `ai.agents` (section 2).
2. Person picks **From**: `Everything` (default), `Today` , `Last 7 days` , `Last 30 days` , or `A date` (date picker, max today). There are **no other range options** (no "last N messages", "unread", "time window", language, length or style choice, or custom prompt: **not implemented**).
3. Tap **Summarise** (disabled while loading, with no agent, or with `A date` and no date chosen). The button text becomes `Reading the chat…` , and after success `Summarise again` .
4. The web sends **one question** to the AI loop (`askWorkspaceAgent`). The text is fixed (`ChatSummaryModal.jsx:33-35`):

```
Summarise my {group chat|private chat} "{chat title}"{, only the messages from YYYY-MM-DD on |
```

5. The model is given tools and calls `read_chat {chat, kind?, since?}` . The **browser** then loads that conversation's history and formats it (section 3.3) and gives it back to the model as the tool result. The model answers from it.
6. The answer text is rendered with `AiMessage` and a **Copy** button. Nothing else.

Up to **4 tool rounds** (`MAX_TOOL_ROUNDS`). If the model keeps calling tools after that: `That took more steps than expected. Please try asking again.`

3.2 Actions and events involved

There is **no summarise action**. The calls are:

STEP	CALL	NOTES
agents	`/sd ai.agents`	section 2
read the chat	`/history <scope> <target>` (a socket text line, not `sd`) → event `{"type":"history","scope","with"|"group"|"host","list":[...]}`	`scope` is `dm` , `group` , `host` or `global` ; the web waits at most 8 s then gives up with `The chat "<name>" did not load in time. Please try again.` With a device cache it first replays the cache and asks `/history <scope> <target> delta <lastId> <now>`
(a person with no open chat yet)	`/sd users.search {"q": "<username|email|mobile>"}`	only when the model names someone not in the chat list
ask the AI	`/sd assistant.relay {...}` (lane "slow", `core/sd_lane.erl:37`)	section 3.4

more context the web adds	`getFeed` (HTTP, notifications), `task.list` , `assoc.list`	section 5.3

There is **no HTTP route** for summarising and **no event stream**.

3.3 The chat as the web hands it to the model

`chatReader.js` `formatTranscript` (this is the useful part to copy):

- Drops deleted messages and anything without a numeric `ts` ; sorts by `ts` ; if `since` is given keeps `ts >= YYYY-MM-DDT00:00:00` (device-local midnight).
- Keeps the **newest 300 messages** (`maxMessages`) and at most **60 000 characters** (`maxChars`), dropping the oldest, never below 20 lines. Each message text is cut to **600 characters** (`oneLine`).
- One line per message, with the time in the viewer's local time:

```
CHAT: group Sales - 3 messages, 2026-11-09 09:12 to 2026-11-09 09:20 (the whole conversation).
[2026-11-09 09:12] ravi: Shall we send the quote today?
[2026-11-09 09:15] meera (you) (reply to ravi: "Shall we send the quote today?"): Yes, after lu
[2026-11-09 09:20] ravi: [form submission] Leave request #41 (Pending) - From: 2026-11-12; To:
```

- If truncated the header says `(only the newest 120 of 540 are shown: older messages were not read)` and the prompt tells the model to say so. If empty: `The group Sales has no messages since 2026-11-01.`
- Form submissions posted in the chat (`kind: "form"`) become `[form submission] <caption> #<id> (<status>) - <caption>: <value>; ...` (each value cut to 200 characters).
- Which chat the model means is resolved **by name** on the client (`resolveChat`): exact name or username, then starts-with, then contains-all-words; `ambiguous` returns `More than one chat matches "<q>": A, B. Ask the user which one.` `none` returns `No chat of yours matches "<q>". Chats: ...` . This is a weakness (section 10).
- The transcript is also kept as the "chat in focus" for follow-up questions, but that only matters in the AI tab.

3.4 The relay call (request and response)

```
/sd assistant.relay {"agent":"claude","payload":{...provider's own request body...},"model":"cl
```

FIELD	MEANING
`agent`	agent `id` from `ai.agents` (optional when the person may use exactly one)

`payload`	the **provider's own JSON body** (must be an object). OpenAI-compatible: `{model, temperature, messages, tools?}` . Anthropic: `{model, max_tokens, temperature, system, messages, tools?}` . Gemini: `{systemInstruction, contents, generationConfig, tools?}`
`model`	optional; used only if `modelChoice` is true for the agent and the model is in the provider's own list for this key (`ai.models {agent}` lists them); otherwise ignored
`timeoutMs`	optional, 5000 to 90000 (default 20000). The web does not send it; it waits 40 s on its side (`RELAY_WAIT_MS`)

Server behaviour (`sd_assistant.erl`): removes `stream` (never streams), sets `model` to the agent's model, checks the 200 KB request cap, applies rate limits, posts to the provider with the agent's key, and writes an audit line (`assistant.relay` , provider, HTTP status, milliseconds: never the content or the key). If the chosen model fails with 404, 429, 5xx, or a 400/403 that mentions "model", it **falls back** to up to **2 more** models of the same provider (never on 401, timeout, or unreachable).

Success envelope:

```
{"type":"sd","action":"assistant.relay","reqId":12,"ok":true,"data":{
  "status":200,
  "body":{ "content":[{"type":"text","text":"**Summary**\n- ..."}], "...": "the provider's own
  "modelUsed":"claude-haiku-5-5",
  "fellBack":false
}}
```

`ok:true` only means the server reached the provider. **Always check** `data.status` : 2xx is success; otherwise classify `data.status + data.body` as in section 3.6. Where the text is:

PROVIDER	TEXT IN `DATA.BODY`	TOOL CALL (IF YOU USE TOOLS)
`anthropic`	join `content[].text` where `type=="text"`	`content[]` items of `type:"tool_use"`
`gemini`	join `candidates[0].content.parts[].text`	parts with `functionCall`
all others (OpenAI-compatible)	`choices[0].message.content`	`choices[0].message.tool_calls`

3.5 Limits

LIMIT	VALUE	WHERE

Request body	200 000 bytes (`too_large`)	`sd_assistant.erl` ? `MAX_REQUEST`
Provider answer	2 000 000 bytes (`too_large` : "The provider's answer was too large.")	`?MAX_ANSWER`
Time per provider call	20 s default; up to 90 s with `timeoutMs`	`?TIMEOUT_MS` , ? `MAX_TIMEOUT_MS`
Calls per person	30 per 60 s	`sd_db:rate("ai:u:<user>", 30, 60)`
Calls per agent (whole org)	600 per 60 s	`sd_db:rate("ai:a:<id>", 600, 60)`
Fallback models	2 extra	`?FALLBACK_TRIES`
Output tokens	`max_tokens: 1800` for Anthropic; not set for the others	`workspaceAgent.js:383`
Temperature	0	same
Messages read	300 / 60 000 chars / 600 per message	`chatReader.js`
Cost / credits	**not metered by Connectum**; billed to the organisation's own provider account	

3.6 Cancellation

Not implemented . Closing the window (×) does not abort the call; the request runs to completion and is wasted. There is no cancel action on the server either. A mobile client can ignore a late answer.

3.7 Errors

Client-side result shape: `{ok:false, code, error, needsKey?, needsTopUp?, needsAdmin?}` .

From our server (`error.code` of the `sd` envelope), mapped by `classifyRelayError` :

SERVER CODE	MEANING	WEB TEXT
`no_agent`	the person may use no agent	`No AI agent is set up for you yet. Ask your administrator.`
`no_access`	agent exists, not allowed or disabled	`You do not have access to this AI agent. Ask your administrator.`
`not_found`	agent id unknown	`That AI agent no longer exists.`
`invalid`	`payload` not an object, no	`payload must be an object. / Choose`

	agent chosen among several, no provider address	`which AI agent to ask.` / `No provider address is set for this agent.`
`key_unreadable`	stored key cannot be decrypted	`This agent's key cannot be read. Tell your administrator.` (web shows `The stored key could not be read. Add it again.`)
`rate_limited`	30/min person or 600/min agent	`Too many questions in a minute. Try again shortly.` (agent-wide: `The AI is busy right now. Try again in a moment.`)
`too_large`	request or answer too big	web: `That question came with too much data. Ask something narrower.`
`upstream_timeout`	provider too slow	`The AI provider took too long to answer. Try again.`
`upstream_unavailable`	cannot reach provider	`Can't reach the AI provider right now. Check the connection and try again.`
`forbidden_address`	custom provider address refused	`The provider address is not allowed. Check the address in the key settings.`
`timeout`, `disconnected`, `not_connected`, `test_timeout`	our socket/server unreachable	`Can't reach Connectum right now. Check the connection and try again.`
anything else		`error.message` or `Something went wrong. Please try again.`

From the provider (`data.status`, classified by `classifyProviderError`):

STATUS / BODY	CODE	WEB TEXT
402, or body matches insufficient quota / billing / credit	`no_credit`	`Your AI provider account has no credit left. Add credit on the provider's billing page, then ask again. Your key is fine.`
401, or 400/403 with invalid/expired/revoked key words	`bad_key`	`The AI provider no longer accepts this key (it may have been rotated, revoked or expired). Add the new key to continue.`
other 403	`bad_key`	`The AI provider refused this key (it may not be allowed to use this model, or it was revoked). Add a key that has access.`

429	`rate`	`The provider is limiting how fast you can ask. Wait a minute and try again.`
404, or 400 mentioning model not found	`model`	`That model is not available to this key. Open the key settings and choose a model your account can use (or clear it to use the default).`
413	`too_large`	`That question came with too much data. Ask something narrower.`
502 / 503 / 504 / 529 / other 5xx	`busy`	`The AI provider is busy or down right now. Try again in a moment.`
other 400	`bad`	`The AI provider could not use that request: <first 160 chars of provider message>`
anything else	`other`	`The AI provider returned an error (HTTP <n>).`

Other texts: empty answer → `The assistant could not answer that.` ; unexpected exception → `Something went wrong reaching the assistant. Please try again.` ; window fallback `The summary could not be made.` Note: for a non-admin, `needsKey` texts say "add a key", which they cannot do (the web text is written for admins; mobile should say "Tell your administrator").

3.8 Summary text format

The model is told (`AI_STYLE_GUIDE` , `aiFormat/aiStyleGuide.js`) to use short paragraphs, bullet lists for 3+ parallel points, Markdown tables for comparisons. The web renders the text with `AiMessage` (`aiFormat/AiMessage.jsx` , "markdown-ish", rendered safely). So the result is **Markdown-style plain text**, not JSON. Exact supported syntax: **UNSURE** ; the care-thread prompt explicitly asks for `**bold**` , `-` bullets, `###` headings and Markdown tables. Treat the text as untrusted: do not render raw HTML or follow links automatically.

---

4. Share in chat

Not implemented . The summary window has one button, **Copy** (`navigator.clipboard.writeText(text)`). The code comment states it: "Nothing is sent to the chat or to anyone; the answer stays on this person's screen" (`ChatSummaryModal.jsx:8`).

Consequently, for the web today:

- No message is posted; no special message type, no "ai"/"summary" marker, no AI sender

identity exists for chat messages.
- Nothing to render for other clients; nothing to edit, delete or forward; nothing stored; recipients cannot see that a summary was made.
- The care-thread summary card is the same: "Only you can see this" (`CareSummaryCard.jsx:61`), it "is never sent to the server or to anyone else".

If mobile adds "share", the only mechanism that exists is to send the summary as an **ordinary text message from the user** (a normal send; it will not be marked as AI). That would be a new product decision (section 10).

---

5. Privacy and end-to-end encryption

5.1 End-to-end encrypted chats

- Direct and group chats are end-to-end encrypted when E2E is on (`E2EBadge.jsx:10-14` : global (`room-general`), host and workspace chats are **not** encrypted).
- The summary code has **no E2E check**: not blocked, no warning, no consent text, no notice to the other people (`ChatSummaryModal.jsx` , `workspaceAgent.js` , `chatReader.js` contain no E2E logic).
- Mechanism: the socket layer decrypts history on the device before the listeners see it (`connectumSocket.js:79` , `decryptEvent` , `needsDecrypt`). `read_chat` takes that decrypted list, so the **plain text goes into the relay request body** and from our server to the AI provider. Messages that could not be opened appear as `🔒 This message is damaged and could not be opened.` and are summarised as such.
- So an E2E chat summary **does leave the device in plain text**, through the server (which sees the request body in memory; it logs only provider, status and time) to a third-party provider chosen by the administrator.
- `docs/PRIVATE_AI_SUMMARY.md` is a **proposal, "Nothing here is built"**: on-device model, consent preview, notice to others. None of it exists. Do not describe it to users as current behaviour.

5.2 What the web shows the user about privacy

Nothing at all in the summary window: no preview of what will be sent, no consent, no provider name (the agent's `name` chip appears only when there are 2 or more agents). **not implemented** .

5.3 What leaves the server to the AI provider

Because the chat summary goes through the general assistant loop (`askWorkspaceAgent`), the provider receives much more than the chat. The system prompt (`systemPrompt` , `workspaceAgent.js:261-319`) contains:

- the chat transcript (above), once the tool has read it;
- `WORKSPACE DATA` : a short summary of the person's own **notifications and tasks** (`buildCompactContext` , up to 120 items fetched, cached 30 s);
- `YOUR CHATS` : the names and kinds of up to 40 of their chats;
- the list of connected patients (usernames and names), if any;
- tool definitions: Data Bin report tools, enterprise form tools, `find_notifications` , `search_workspace` (if RAG is configured), `read_chat` , `list_chats` , patient tools, plus the agent plugin's tools when built in. A model could therefore call them in the same run. **UNSURE** whether prompt injection in a chat message could trigger them; the prompt tells the model to answer only from the transcript.
- the person's display name and today's date.

The chat text of **other people** (the other side of a direct chat, the rest of a group) leaves with it.

---

6. Other AI actions inside the chat

6.1 Care / patient thread summary (`room-care-<person>` only)

- Header button `auto_awesome` , "Summary of this thread" (`data-testid="care-summary-open"`).
- Action: `/sd care.summary {"person":"<username>"}` , access `user` , answer is **built from stored data, not written by a model** (`forms/sd_carethread.erl:101-132`): `{person, team, stats:{forms,messages,people,first,last,truncated}, latest, timeline}` . Reads at most **500** messages (`truncated:true` when capped), up to 8 fields per form and 160 characters per value. Forms the asker may not see are left out. Errors: `not_found` `No such user.` / `No shared conversation with that person.`
- Shown as a card **inside the thread** that only the asker sees (Refresh / Copy / Dismiss), kept in memory on that screen (`privateSummaries`).
- **Follow-up questions** in the card go to `askAboutThread` (agent = first of `ai.agents`): it loads the thread history (`/history group care-<username>`) up to **400 messages / 70 000 chars**, builds a Markdown-answer prompt, runs the same relay with **no tools**, and returns `{ok, text}` . The questions and answers are kept in the card (`utils/threadAsk.js` via `storageKey` , **UNSURE** exact storage: it looks like browser storage) and are cleared with Dismiss / "start over". If no agent: `No AI agent is set up for you yet. Ask your administrator.`
- Same E2E position as 5.1 does not apply here: the care thread is a server-side group; **UNSURE** whether its messages are E2E.

6.2 `#<alias> <question>`

Opens the AI tab and asks there. Out of scope

Opens the AI tab and asks there. Out of scope.

6.3 Rewrite, translate, suggest a reply, ask AI about this chat (free-form), AI on a single message

Not implemented in the chat window (searched `chat-app/features` for translate, rewrite, suggested reply and "improve"; the only hits are CSS animations and unrelated UI). Free-form questions about a chat exist only in the AI tab (`read_chat` tool).

---

7. Feed, notifications, unread, persistence

- **Feed items / notifications / push for AI activity: none.** `sd_assistant.erl` writes only an audit line; nothing is created for the asker or anyone else.
- **Unread counts:** unaffected; no message is created.
- **History of summaries:** none. The chat summary lives in React state of the open window and is lost on close. It has no id and cannot be fetched again. Doing it again is a new, paid provider call.
- `ai.conv.list/get/save/rename/pin/delete` store the AI **tab's** conversations (sealed, per person); the summary window does **not** write to them (no call in `ChatSummaryModal.jsx`). Out of scope.
- The only live event related to AI is `ai_agents_changed` (section 2.1).

---

8. Auth and permission rules for mobile

RULE	DETAIL
Sign-in	`ai.agents` , `assistant.relay` , `ai.models` are `user` level: any signed-in person
Agent access	`enabled` and permission `ai:<id>` (or the older `access` rule until one is set). Checked on **every** relay call
Chat membership	**The server does not check it for summaries**, because it never sees "which chat": the client sends the text. The check happens when the client loads history (`/history ...`), which the server limits to the person's own conversations. **A mobile client must only send text it could legitimately read.** Never let a user type an arbitrary chat id into a prompt
Org toggle	none besides per-agent `enabled` and access
Administrators	have no extra rights to summarise; same call. They alone can add keys
Care thread	`care.summary` limits content to what the asker may see (form visibility)
Rate limits	30 calls/min/person, 600/min/agent (shared by the whole organisation)

One session per client kind	a phone and a laptop can be signed in at once; no AI-specific effect

Web vs what mobile may do:

- The web runs the whole loop in the browser because the server only relays. Mobile **may** do the same, but does not need the tool loop (section 10).
- The `agent` argument and `model` argument are the only inputs the server validates. The provider payload is the client's responsibility; the server overwrites `model` and removes `stream`.
- Mobile cannot use the web's per-device history cache; it loads history its own way.

---

9. Test data

9.1 Set up an agent on a test org

1. Sign in as an **administrator** on the web.
2. Admin console > **AI agents** > add: name `Claude`, command `claude`, provider `Anthropic (Claude)`, model (blank uses `claude-haiku-5-5`), key (paste), access "everyone". Press **Test**; success means the key works and credit exists (it sends "Reply with OK").
3. Or over the socket (administrator session). The key is whatever you own; **never commit it**:

```
/sd admin.ai.save {"name":"Claude","alias":"claude","provider":"anthropic","model":"claude-haik
/sd admin.ai.test {"id":"claude","reqId":2}
```

The save answer is `{"agent":{"id":"claude","name":"Claude","alias":"claude","provider":"anthropic","model":"claude-haiku-5-5","baseUrl":"","last4":"****","enabled":true,"modelChoice":true,"updated":<ms>,"access":{...}}}` (the key is never returned; `last4` shows its last four characters).

9.2 Not set up (copy from the server's logic)

```
/sd ai.agents {"reqId":21}
```

```
{"type":"sd","action":"ai.agents","reqId":21,"ok":true,"data":{"agents":[]}}
```

Show: `No AI is set up for you yet. Ask your administrator.` If the app calls `assistant.relay` anyway:

```
{"type":"sd","action":"assistant.relay","reqId":22,"ok":false,"error":{"code":"no_agent","messa
```

9.3 A summarise call

These are **built from the code, not captured from a live run** (no live run was done). The pair is a one-shot request for an Anthropic agent, as the web would send it after reading the chat.

Request (single round, no tools; the transcript is put in the `user` message by the mobile client):

```
/sd assistant.relay {"agent":"claude","payload":{"model":"claude-haiku-5-5","max_tokens":1800,"
```

Response:

```
{"type":"sd","action":"assistant.relay","reqId":31,"ok":true,"data":{
  "status":200,
  "body":{"id":"msg_...","type":"message","role":"assistant","content":[{"type":"text","text":"
  "modelUsed":"claude-haiku-5-5","fellBack":false}}
```